DATA STRUCTURES

UNIT - 1

DATA

- Data is the basic fact or entity that is utilized in calculation or manipulation.
- There are two different types of data Numeric data and Alphanumeric data.
- Data can come in the form of text, observations, figures, images, numbers, graphs, or symbols.
- For example, data might include individual prices, weights, addresses, ages, names, temperatures, dates, or distances.
- Data is a raw form of knowledge and, on its own, doesn't carry any significance or purpose.
- When programmer collects such type of data for processing he would require to store them in computer’s main memory.
- The process of storing data into computer’s main memory is called representation.
- Data to be processed must be organized into particular fashion and these organization leads to structuring of data.

Introduction to Data Structure

- Computer is an electronic machine which is used for data processing and manipulation.
- When programmer collects such type of data for processing, he would require to store all of them in computer's main memory.
- In order to make computer work we need to know:
  - Representation of data in computer.
  - Accessing of data.
  - How to solve problem step by step.
- For doing this task we use data structure.
- Using appropriate data structures can help programmers save a good amount of time while performing operations such as storage, retrieval, or processing of data. Manipulation of large amounts of data is easier.

Introduction to Data Structure

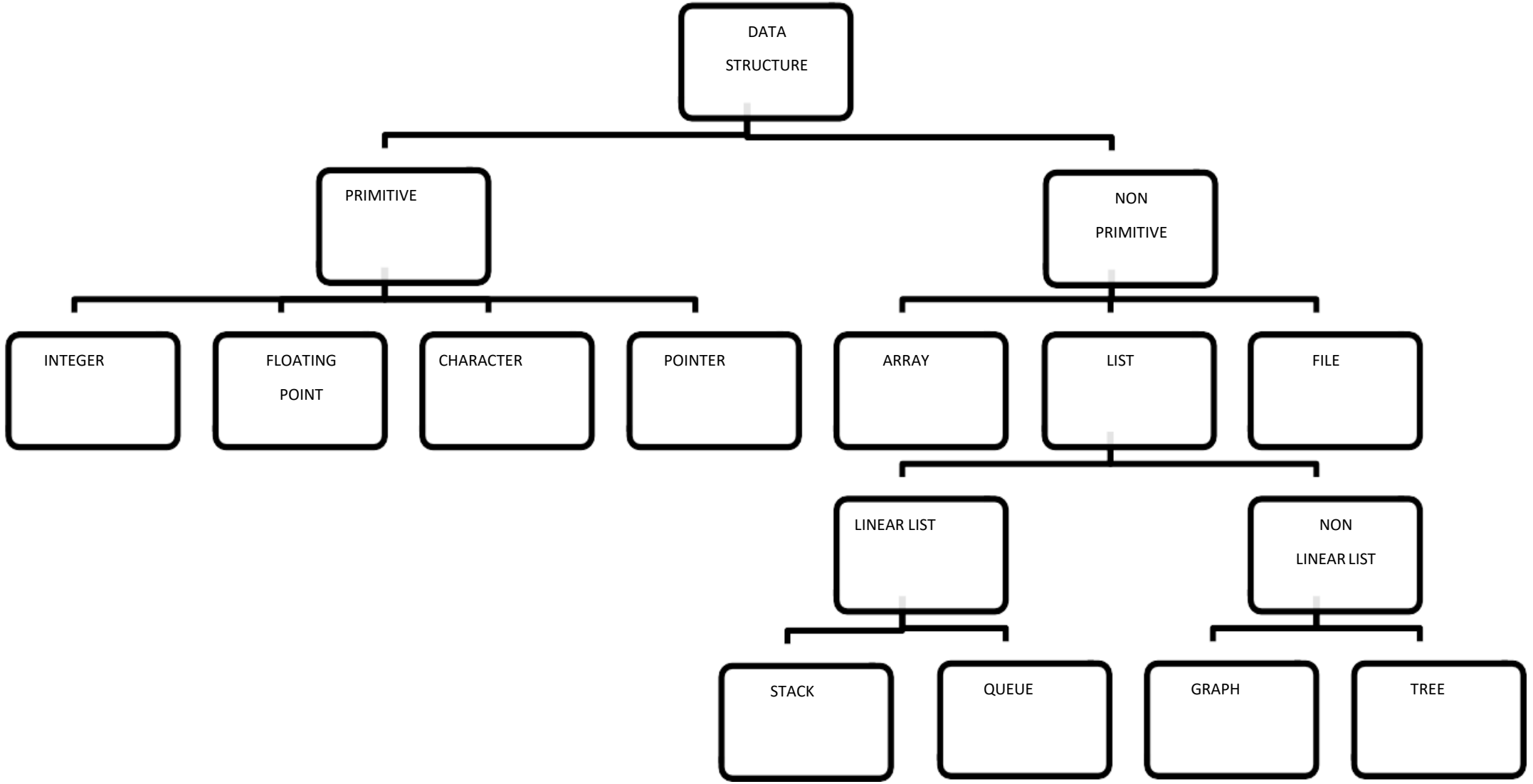
- **Data structure** is a representation of the logical relationship existing between individual elements of data.
- Data Structure is a way of organizing all data items that considers not only the elements stored but also their relationship to each other.
- We can also define data structure as a mathematical or logical model of a particular organization of data items.
- The representation of particular data structure in the main memory of a computer is called as **storage structure.**
- The storage structure representation in auxiliary memory is called as **file structure**.
- It is defined as the way of storing and manipulating data in organized form so that it can be used efficiently.

- **Note**: Auxiliary memory **holds programs and data for future use**, and, because it is nonvolatile (like ROM), it is used to store inactive programs and to archive data. Early forms of auxiliary storage included punched paper tape, punched cards, and magnetic drums.

Data Structure

- Data Structure mainly specifies the following four things
  - Organization of Data
  - Accessing methods
  - Degree of associativity
  - Processing alternatives for information
- Algorithm + Data Structure = Program
- Data structure study covers the following points
  - Amount of memory require to store.
  - Amount of time require to process.
  - Representation of data in memory.
  - Operations performed on the data.

Classification of Data Structure



Data structures

- Data Structures are normally classified into two broad categories

1) Primitive Data Structure
2) Non-primitive data Structure

Data Types:

- A particular kind of data item, as defined by the values it can take, the programming language used, or the operations that can be performed on it.

Primitive Data Structure

- Primitive data structures are basic structures and are directly operated upon by machine instructions.
- They have different representations on different computers.
- Integers, floats, character and pointers are examples of primitive data structures.
- These data types are available in most programming languages as built in type.
  - **Integer**: It is a data type which allows all values without fraction part. We can use it for whole numbers.
  - **Float**: It is a data type which use for storing fractional numbers.
  - **Character**: It is a data type which is used for character values.
  - **Pointer**: A variable that holds memory address of another variable are called pointer.

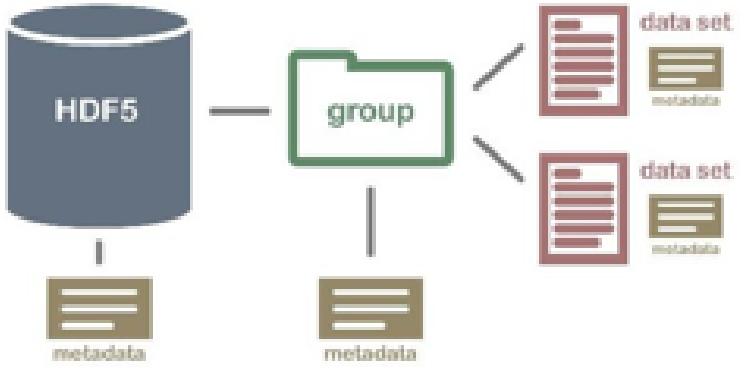
Non primitive Data Type

- These are more sophisticated data structures.
- These are derived from primitive data structures.
- The non-primitive data structures emphasize on structuring of a group of homogeneous (similar) or heterogeneous (dissimilar) data items.
- Examples of Non-primitive data type are Array, List, and File etc.
- A Non-primitive data type is further divided into Linear and Non-Linear data structure
  - **Array:** An array is a fixed-size sequenced collection of elements of the same data type.
  - **List:** An ordered set containing variable number of elements is called as Lists.
  - **File:** A file is a collection of logically related information. It can be viewed as a large list of records consisting of various fields.

Non primitive Data Type

Array

List



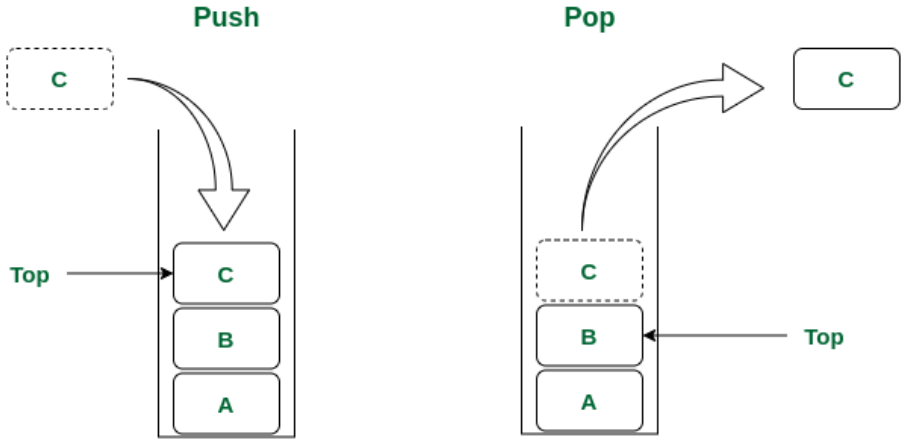
File

Linear data structures

- A data structure is said to be Linear, if its elements are connected in linear fashion by means of logically or in sequence memory locations.
- There are two ways to represent a linear data structure in memory,
  - Static memory allocation –
  - Static data structure has a fixed memory size. It is easier to access the elements in a static data structure. *An example of this data structure is an array.*
  - Dynamic memory allocation –
  - In dynamic data structure, the size is not fixed. It can be randomly updated during the runtime which may be considered efficient concerning the memory (space) complexity of the code. *Examples of this data structure are queue, stack, etc.*
- The possible operations on the linear data structure are: Traversal, Insertion, Deletion, Searching, Sorting and Merging.
- Examples of Linear Data Structure are Stack and Queue.

Stack

- Stack is a data structure in which insertion and deletion operations are performed at one end only.
- The insertion operation is referred to as ‘PUSH’ and deletion operation is referred to as ‘POP’ operation.
- Stack is also called as Last in First out (LIFO) data structure.
- There are many real-life examples of a stack. Consider an example of plates stacked over one another in the canteen. The plate which is at the top is the first one to be removed, i.e. the plate which has been placed at the bottommost position remains in the stack for the longest period of time. So, it can be simply seen to follow LIFO(Last In First Out)/FILO(First In Last Out) order.
- Application of stack in data structure: Evaluation of Arithmetic Expressions, Reverse a Data, Processing Function Calls



Stack Data Structure

Queue

- The data structure which permits the insertion at one end and Deletion at another end, known as Queue.
- End at which deletion occurs is known as FRONT end and another end at which insertion occurs is known as REAR end.
- Queue is also called as First in First out (FIFO) data structure.
- A Queue is like a line waiting to purchase tickets, where the first person in line is the first person served. (i.e. First come first serve).

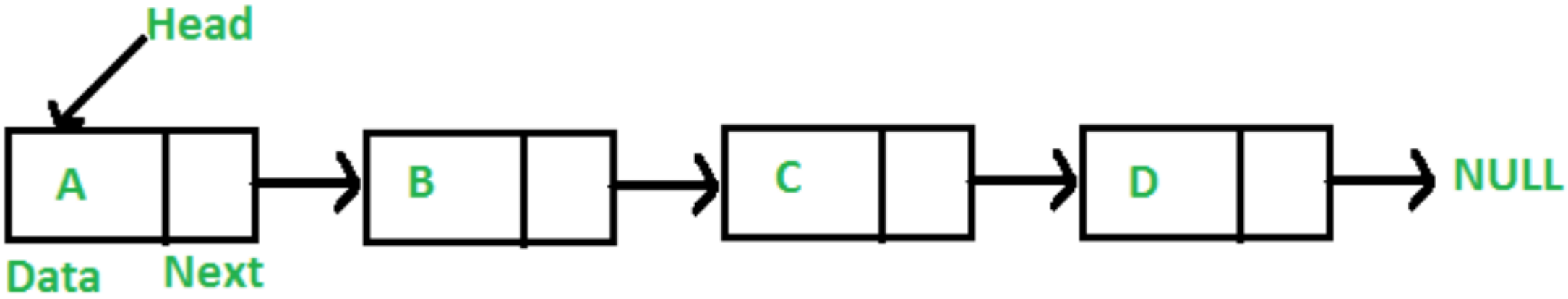
Queue Data Structure

Queue

- **Some of the Application of Queue in Data Structure**
- Here are some of the common application of queue in data structure:
- Queues can be used to schedule jobs and ensure that they are executed in the correct order.
- Queues can be used to manage the order in which print jobs are sent to the printer.
- Queues are used to implement the breadth-first search algorithm.
- Queues can be used to manage incoming calls and ensure that they are handled in the correct order.
- Queues can be used to manage the order in which processes are executed on the CPU.
- Queues can be used for buffering data while it is being transferred between two systems. When data is received, it is added to the back of the queue, and when data is sent, it is removed from the front of the queue.
- Applied as buffers on playing music in the mp3 players or CD players.
- Applied on handling the interruption in the operating system.
- Applied to add a song at the end of the playlist.
- Applied as the waiting queue for using the single shared resources like CPU, Printers, or Disk.

Linked List

- A linked list is a linear data structure, in which the elements are not stored at contiguous memory locations. In simple words, a linked list consists of nodes where each node contains a data field and a reference(link)



Linked List

Types :

- Singly Linked List
- Circular Linked List
- Doubly Linked List
- **Applications of Linked List in Computer Science**
- Representing polynomials, Polynomial manipulation, Implementing stacks and queues, Implementing graphs
- Music players (play list), Image viewers, Web browsers

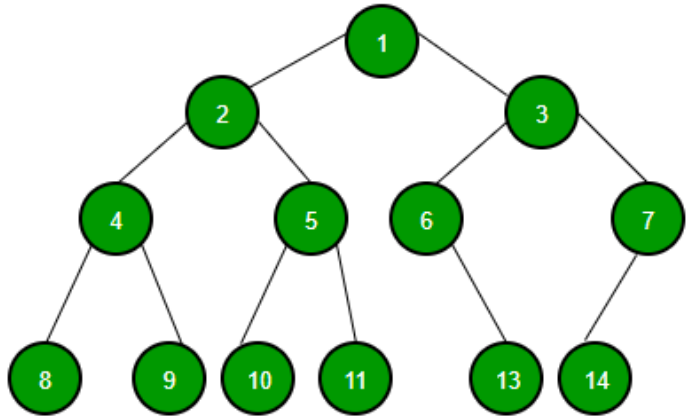
Nonlinear data structures

- Nonlinear data structures are those data structure in which data items are not arranged in a sequence.

- Examples of Non-linear Data Structure are Tree and Graph.

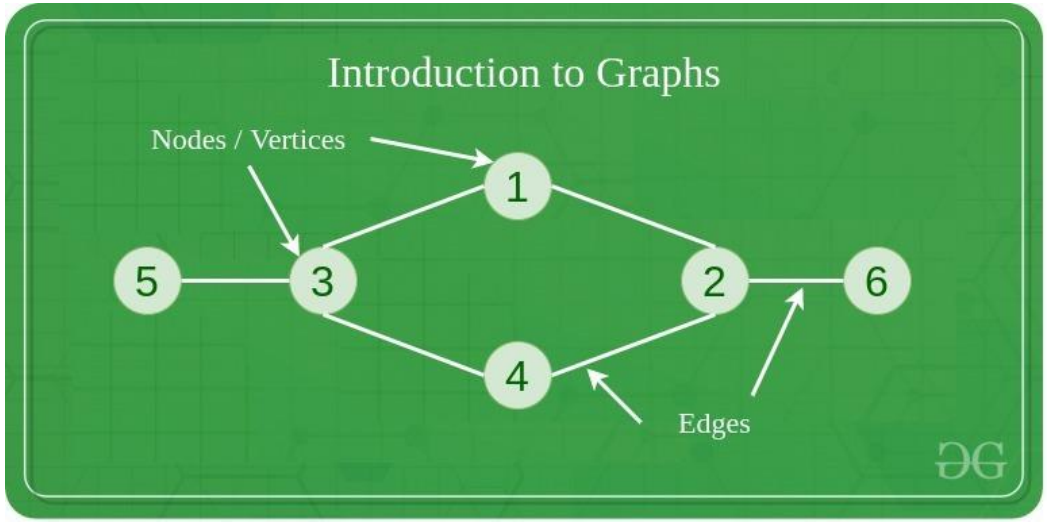
Tree

- A tree can be defined as finite set of data items (nodes) in which data items are arranged in branches and sub branches according to requirement.
- Trees represent the hierarchical relationship between various elements.
- Tree consist of nodes connected by edge, the node represented by circle and edge lives connecting to circle.
- Another example of a tree structure that you probably use every day is a file system. In a file system, directories, or folders, are structured as a tree.



Graph

- A Graph is a non-linear data structure consisting of vertices and edges.
- The vertices are sometimes also referred to as nodes and the edges are lines or arcs that connect any two nodes in the graph.
- More formally a Graph is composed of a set of vertices(V) and a set of edges(E). The graph is denoted by G(E, V).



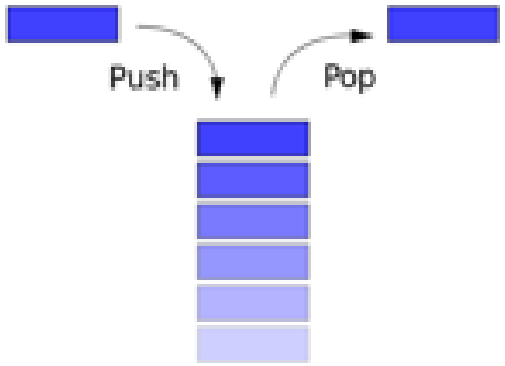
Graph

- Graphs are used to solve many real-life problems.
- Graphs are used to represent networks.
- The networks may include paths in a city or telephone network or circuit network. Graphs are also used in social networks like linkedIn, Facebook.
- For example, in Facebook, each person is represented with a vertex(or node). Each node is a structure and contains information like person id, name, gender, locale etc.
- A tree can be viewed as restricted graph.
- Graphs have many types:
    - Un-directed Graph
    - Directed Graph
    - Mixed Graph
    - Multi Graph
    - Simple Graph
    - Null Graph
    - Weighted Graph

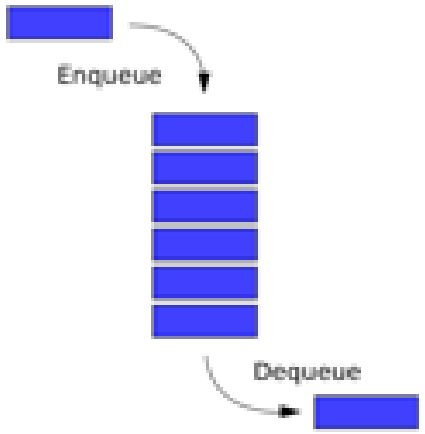
Difference between Graph and Tree

No.	Graph	Tree
1	It is a non-linear data structure.	It is also a non-linear data structure.
2	A graph is a set of vertices/nodes and edges.	A tree is a set of nodes and edges.
3	In the graph, there is no unique node which is known as root.	In a tree, there is a unique node which is known as root.
4	Each node can have several edges.	Usually, a tree can have several child nodes, and in the case of binary trees, each node consists of two child nodes.
5	Graphs can form cycles.	Trees cannot form a cycle.

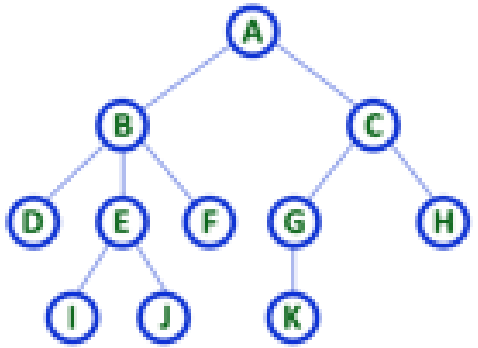
Linear – Non Linear Data types



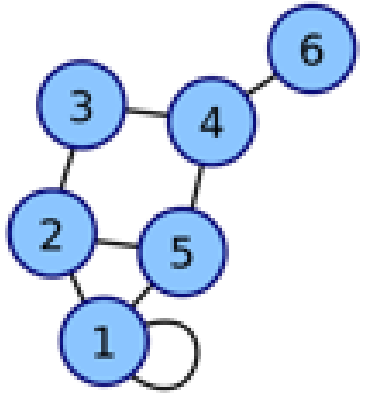
Stack



Queue



Tree



Graph

Difference between Linear and Non Linear Data Structure

Linear Data Structure	Non-Linear Data Structure
Every item is related to its previous and next items.	Every item is attached with many other items.
Data is arranged in linear sequence.	Data is not arranged in sequence.
Data items can be traversed in a single run.	Data cannot be traversed in a single run.
Eg. Array, Stacks, linked list, queue.	Eg. tree, graph.
Implementation is easy.	Implementation is difficult.

Operation on Data Structures

- Design of efficient data structure must take operations to be performed on the data structures into account. The most commonly used operations on data structure are broadly categorized into following types

1) Create

- The create operation results in reserving memory for program elements. This can be done by declaration statement. Creation of data structure may take place either during compile-time or run-time. malloc() function of C language is used for creation.

2) Destroy

- Destroy operation destroys memory space allocated for specified data structure. free() function of C language is used to destroy data structure.

Operation on Data Structures

3) Selection

- Selection operation deals with accessing a particular data within a data structure.

4) Updation

- It updates or modifies the data in the data structure.

5) Searching

- It finds the presence of desired data item in the list of data items, it may also find the locations of all elements that satisfy certain conditions.

Operation on Data Structures

6) Sorting

- Sorting is a process of arranging all data items in a data structure in a particular order, say for example, either in ascending order or in descending order.

7) Merging

- Merging is a process of combining the data items of two different sorted list into a single sorted list.

8) Splitting

- Splitting is a process of partitioning single list to multiple list.

9) Traversal

- Traversal is a process of visiting each and every node of a list in systematic manner.

Time and space analysis of algorithms

Algorithm:

- An essential aspect to data structures is algorithms.
- Data structures are implemented using algorithms.
- An algorithm is a procedure that you can write as a C function or program, or any other language.
- An algorithm states explicitly how the data will be manipulated.

Algorithm Efficiency

- Some algorithms are more efficient than others.
- We would prefer to choose an efficient algorithm.
- The complexity of an algorithm is a function describing the efficiency of the algorithm in terms of the amount of data the algorithm must process.
- Usually there are natural units for the domain and range of this function.
- There are two main complexity measures of the efficiency of an algorithm

1) Time complexity - a measure of amount of time for an algorithm to execute.

2) Space Complexity - a measure of the amount of memory needed for an algorithm to execute.

Space complexity

- Whenever we say that our algorithm is sufficient then it means that the algorithm is solving the problem in less amount of time while taking the least amount of space.
- Let's take an example of sorting algorithms like insertion and heap sort doesn't creates a new array during sorting as they are in-place sorting techniques but merge sort creates an array during sorting of elements which takes an extra space so if there is a concern of space then obviously one will prefer the insertion or heap sort.
- Suppose you are provided with a task to sort the given array, so there are many sorting algorithms but you will choose the optimized and efficient one always and for choosing that you have to analyze different algorithms on the basis of their space and time complexity.
- So as we know that analyzing the algorithm is a much-needed task after designing an algorithm so as to increase the efficiency of an algorithm.
- During analyzing any problem or algorithm you all may have encountered time complexity and space complexity.

Space complexity

- Space complexity is nothing but the amount of memory space that an algorithm or a problem takes during the execution of that particular problem/algo.
- The space complexity is not only calculated by the space used by the variables in the problem/algo it also includes and considers the space for input values with it.
- There is a sort of confusion among people between the space complexity and the auxiliary space.
- So let's be clear about that, so auxiliary space is nothing but the space required by an algorithm/problem during the execution of that algorithm/problem and it is not equal to the space complexity because space complexity includes space for input values along with it also.
- So we can say that space complexity is the combination or sum up of the auxiliary space and the space used by input values.
- **Space Complexity = Auxiliary Space + Space used for input values**
- Space complexity depends on variables, data structures, allocations, function call

Time complexity

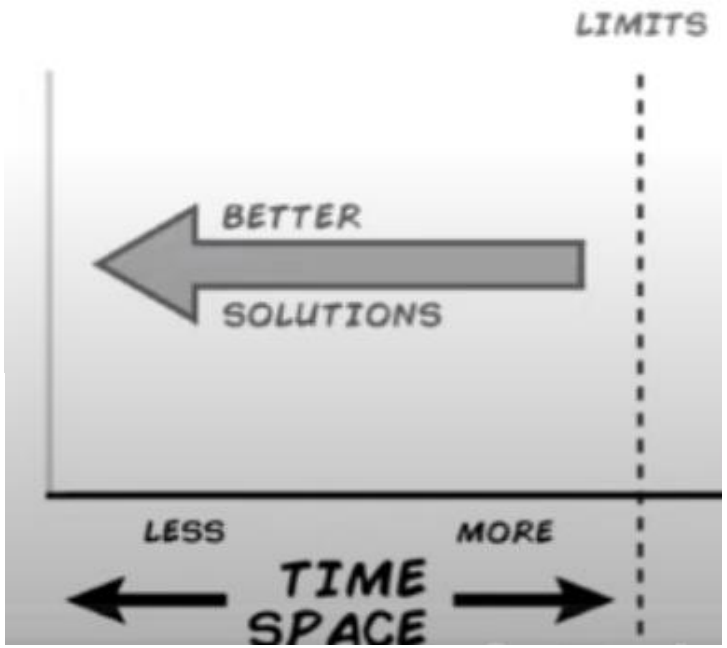
- **Time Complexity** is a function describing the amount of time an algorithm takes in terms of the amount of input to the algorithm.
- "Time" can mean the number of memory accesses performed, the number of comparisons between integers, the number of times some inner loop is executed, or some other natural unit related to the amount of real time the algorithm will take.

- So time complexity depends on operations, comparisons, loops, pointer reference, function call to outside.

Time complexity

The time taken for an algorithm is comprised of two times:

1. Compilation Time
2. Run Time



➢ **Compile Time**

- Compilation time is the time taken to compile an algorithm.
- While compiling it checks for the syntax and semantic errors in the program and links it with the standard libraries.

➢ **Run Time**

- It is the time to execute the compiled program.
- The run time of an algorithm depend upon the number of instructions present in the algorithm.
- Note that run time is calculated only for executable statements and not for declaration statements

Time complexity

Time complexity of an algorithm is generally classified as three types.

- Also called as Asymptotic Analysis & Asymptotic Notations .

1. Worst case(longest time) Big O (upper bound)
2. Average case (average time) Theta (tighter bound)
3. Best Case(shorter time) Omega (lower bound)

Time complexity

Worst Case: It is the longest time that an algorithm will use over all instances of size n for a given problem to produce a desired result.

Average Case: It is the average time(or average space) that the algorithm will use over all instances of size n for a given problem to produce a desired result.

Best Case: It is the shortest time (or least space) that the algorithm will use starting instances of size n for a given problem to produce a desired result.

Time complexity

Best case efficiency

- let we have to find 25 in List =>25,31,42,71,105
- k=25
- 25 is present at the first position
- Since only single comparison is made to search the element so we say that it is best case efficiency
- $C_{Best}(n)=1$

Time complexity

Worst case efficiency

- If we want to search the element which is present at the last of the list or not present at all in the list then such cases are called the worst case efficiency.
- let we have to find 105 in List =>25,31,42,71,105
- k=105
- Therefore we have to make 5 (=n) comparisons to search the element
- C_{Worst} (n)=n
- And if we have to find 110
- k=110
- Since the element is not in the list even then we have to make 5 (=n) comparisons to search the element
- C_{Worst} (n)=n

Time complexity

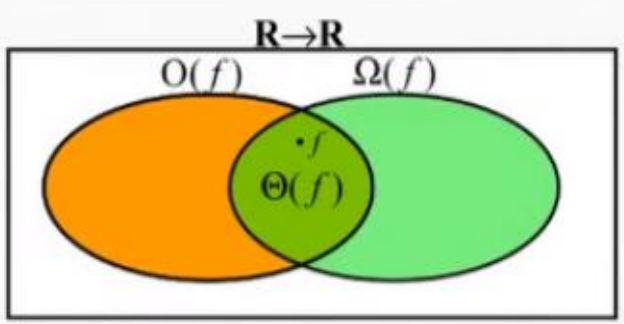
Average case efficiency

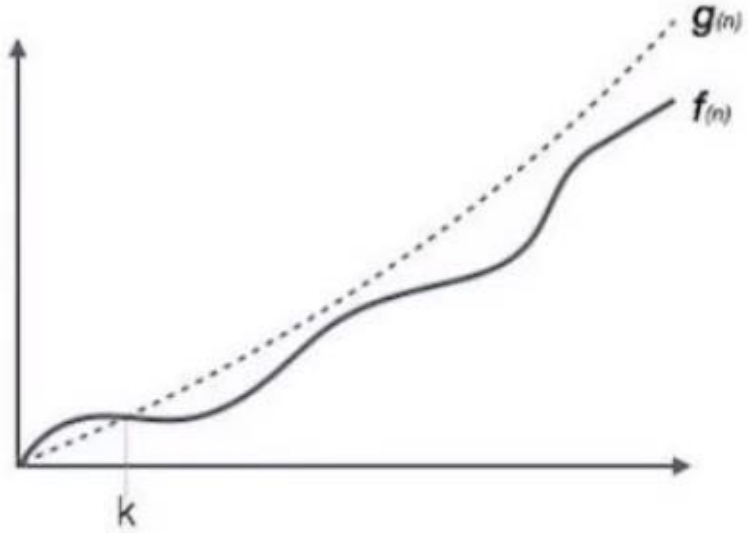
- Let the element is not present at the first or the last position
- Let it is present somewhere in middle of the list
- We know that probability of a successful search = p where $0 \leq p \leq 1$.
- Probability of an unsuccessful search =1-p

Time complexity

- Following are the commonly used asymptotic notations.
- It calculate the running time complexity of an algorithm.

1. **O (Big Oh) Notation**
2. **Ω (Omega) Notation**
3. **θ (Theta) Notation**





1. Big Oh Notation, O

- The notation O(n) is the formal way to express the **upper bound** of an algorithm's running time.
- It measures the **worst case** time complexity or the **longest amount of time** an algorithm can possibly take to complete.

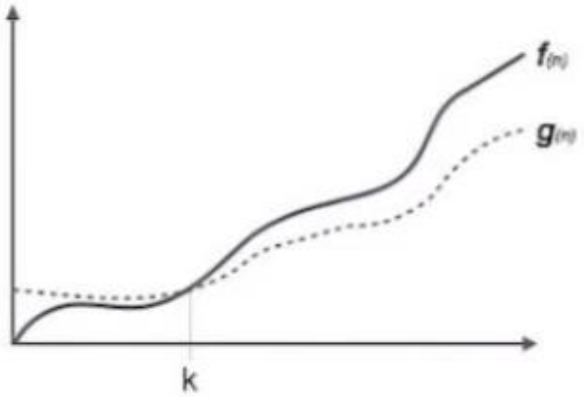
$$O(f(n)) = f(n) \leq c.g(n)$$

Time complexity

2. Omega Notation, Ω

- The notation Ω(n) is the formal way to express the **lower bound** of an algorithm's running time.
- It measures the **best case** time complexity or the best amount of time an algorithm can possibly take to complete.

$$\Omega(f(n)) = g(n) \leq c.f(n)$$



3. Theta Notation, θ

- The notation θ(n) is the formal way to express **both the lower bound and the upper bound** of an algorithm's running time with **Average case** time complexity .
- It is represented as follows −

$$\theta(f(n)) = O(f(n)) \text{ and } g(n) = \Omega(f(n))$$

Time complexity

Following is a list of some common asymptotic notations –

constant	–	$O(1)$
logarithmic	–	$O(\log n)$
linear	–	$O(n)$
n log n	–	$O(n \log n)$
quadratic	–	$O(n^2)$
cubic	–	$O(n^3)$
polynomial	–	$n^{O(1)}$
exponential	–	$2^{O(n)}$

Time complexity

- Example: Linear Search Complexity

- Best Case : Item found at the beginning: ***One comparison***
- Worst Case : Item found at the end: ***n comparisons***
- Average Case :Item may be found at index 0, or 1, or 2, . . . or n - 1
  - Average number of comparisons is: $(1 + 2 + \ldots + n) / n = (n+1) / 2$

- Worst and Average complexities of common sorting algorithms

Method	Worst Case	Average Case	Best Case
Selection Sort	n^2	n^2	n
Insertion Sort	n^2	n^2	n
Merge Sort	nlogn	nlogn	nlogn
Quick Sort	n^2	nlogn	nlogn

Time complexity

- 1
- log*n*
- *n*
- *n* log *n*
- n^2
- 2^n
- *n*!

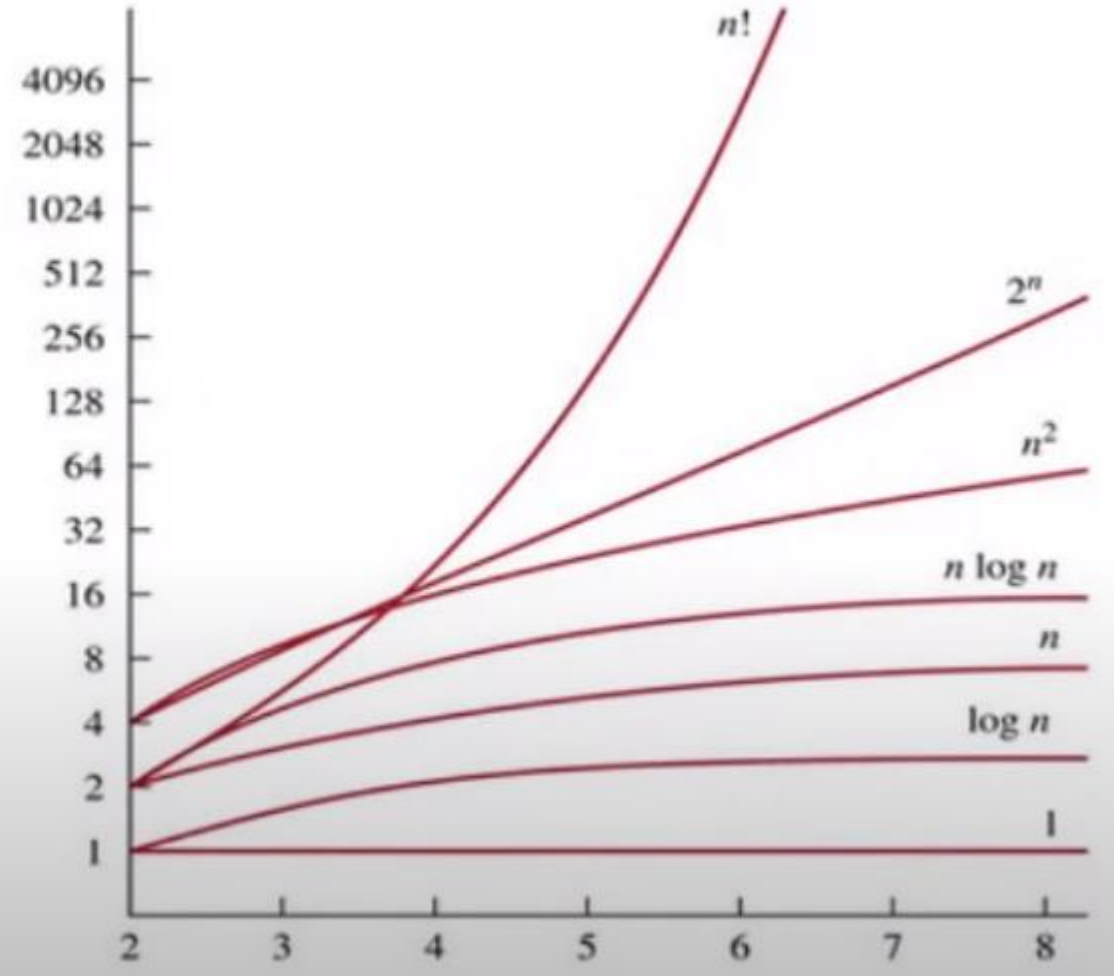


FIGURE 3 A Display of the Growth of Functions Commonly Used in Big-*O* Estimates.

Time complexity

- **Q. Imagine a classroom of 100 students in which you gave your pen to one person. You have to find that pen without knowing to whom you gave it.**
- Here are some ways to find the pen and what the O order is.
- **$O(n^2)$:** You go and ask the first person in the class if he has the pen. Also, you ask this person about the other 99 people in the classroom if they have that pen and so on,
  This is what we call $O(n^2)$. // nested loop
- **O(n):** Going and asking each student individually is O(N). // single loop
- **O(log n):** Now I divide the class into two groups, then ask: “Is it on the left side, or the right side of the classroom?” Then I take that group and divide it into two and ask again, and so on. Repeat the process till you are left with one student who has your pen. This is what you mean by O(log n).

Time complexity

- I might need to do:
- The **O(n^2)** searches if **only one student knows on which student the pen is hidden**.
- The **O(n)** if **one student had the pen and only they knew it**.
- The **O(log n)** search if **all the students knew**, but would only tell me if I guessed the right side.
- Note:***Time Complexity considers how many times each statement executes.***

Big O Notation

- Big O Notation is **a tool used to describe the time complexity of algorithms**.
- It calculates the time taken to run an algorithm as the input grows.

Time complexity

1)Example:

print("Hello World") - > **O(1)**

2) Example:

```
int i, n = 8;
for (i = 1; i <= n; i++) {
        printf("Hello World !!!\n");
}
```

Complexity: O(n)

Time complexity

```cpp
#include<bits/stdc++.h>
using namespace std;
int main( ){
  cout<<"Hello dev 1\n";
  cout<<"Hello dev 2\n";
  cout<<"Hello dev 3\n";
}
```

- There are three separate statements inside the main() function, the overall time complexity is **O(1)** because each statement only takes unit time to complete execution.

Time complexity

```cpp
#include<bits/stdc++.h>
using namespace std;
int main(){
  int i,n=35;
  for(i=1;i<=n;i++){
    cout<<"Hello dev 1\n";
    cout<<"Hello dev 2\n";
    cout<<"Hello dev 3\n";
  }
}
```

Here, the **O(1)** chunk of code (the 3 cout statements) is enclosed inside a looping statement which **repeats iteration for 'n' number of times**. Thus our overall time complexity becomes n*O(1), i.e., **O(n)**.

Time complexity

You could say that when an algorithm employs statements that are only executed once, the time required is constant. However, when the statement is in loop condition, the time required grows as the number of times the loop is set to run grows.

When an algorithm has a mix of single-executed statements and LOOP statements, or nested LOOP statements, the time required increases according to the number of times each statement is run.

Time complexity

Sum(a, b) { return a + b }

Time Complexity = O(1)

Time complexity

```
total =0                    // no of times=1
for i=0 to n-1              // no of times=n+1 (+1 for the end false condition)
    sum = sum + A[i]        // no of times=n
return sum                  // no of times = 1
```

Time complexity = $O(n)$

Day 1 Programs

- **Sum of digits algorithm**
- To get sum of each digits by c program, use the following algorithm:
- Step 1: Get number by user
- Step 2: Get the modulus/remainder of the number
- Step 3: sum the remainder of the number
- Step 4: Divide the number by 10
- Step 5: Repeat the step 2 while number is greater than 0.

Day 1 Programs

- **Write a program to find the total of each rows and columns in a given matrix.**

```
Input: array[4][4] = { {1, 1, 1, 1},
                       {2, 2, 2, 2},
                       {3, 3, 3, 3},
                       {4, 4, 4, 4}};
Output: Sum of the 0 row is = 4
        Sum of the 1 row is = 8
        Sum of the 2 row is = 12
        Sum of the 3 row is = 16
        Sum of the 0 column is = 10
        Sum of the 1 column is = 10
        Sum of the 2 column is = 10
        Sum of the 3 column is = 10
```

Day 1 Programs

- **Algorithm to find the total of each rows and columns in a given matrix.**
- Algorithm

- **STEP 1:** START
- **STEP 2:** DEFINE rows (m) , cols (n), sum
- **STEP 3:** input the order of matrix // enter m and n
- **STEP 4:** input the coefficient of the matrix // for loop nested : outer loop i=0;i<m (rows) & inner loop j=0;j<n (cols)
- **STEP 5:** calculate sum of the row // nested for loop :outer loop i=0;i<m (rows) & inner loop j=0;j<n (cols) {sum=sum+a[i][j]}
- **STEP 5:** Print sum
- **STEP 6:** Set sum =0
- **STEP 7:** calculate sum of the col// nested for loop :outer loop i=0;i<n (cols) & inner loop j=0;j<n (rows) {sum=sum+a[i][j]}
- **STEP 8:** Print sum
- **STEP 9:** Set sum =0
- **STEP 10:** END

Day 1 Programs

Find second highest element from array. (23,12,43,40)

1. Input size and elements in array, store it in some variable say size and arr.
2. Declare two variables max1 and max2 to store first and second largest elements. Store minimum integer value in both i.e. max1 = max2 = INT_MIN.

Note: INT_MIN **specifies that an integer variable cannot store any value below this limit**. Values of INT_MAX and INT_MIN may vary from compiler to compiler. Following are typical values in a compiler where integers are stored using 32 bits. Value of INT_MAX is +2147483647. Value of INT_MIN is -2147483648.

3. Iterate though all array elements, run a loop from 0 to size - 1. Loop structure should look like for(i=0; i<size; i++).

4. Inside loop, check if current array element is greater than max1, then make largest element as second largest and current array element as largest. Say, max2 = max1 and max1 = arr[i].

5. Else if the current array element is greater than max2 but less than max1 then make current array element as second largest i.e. max2 = arr[i].